Android Application Forensic Assessment PeriodTracker (Flo Health) — Static Code Analysis & Live Traffic Profiling

Application: org.iggymedia.periodtracker, version 4.13.0

Analysis Environment: Windows 11 VM (VMware Fusion · macOS Tahoe host) + Physical Android Device

Examiner: **Alister A. Rodrigues**

Tools: JADX-GUI · Apktool · PCAPdroid

Examination Date: November 2025

Introduction

This assessment examines the PeriodTracker APK (Flo Health, org.iggymedia.periodtracker v4.13.0) through both static and dynamic analysis to characterize the application's behavior, permission model, third-party dependencies, and live network activity. PeriodTracker was selected for its comparatively self-contained feature scope relative to high-complexity communication applications, enabling cleaner isolation of code behavior and network requests.

Static analysis was conducted on a Windows 11 virtual machine (VMware Fusion on macOS) using JADX-GUI and Apktool to inspect the decompiled application structure, AndroidManifest.xml, declared permissions, third-party SDK references, and embedded resource strings. Dynamic analysis was performed on a physical Android device using PCAPdroid to capture live network traffic generated during application execution, providing a real-device traffic profile against the static findings. All steps were executed in a controlled, reproducible environment with findings documented through screenshots and traffic captures.

Methodology

The following methodology was applied to examine the PeriodTracker APK using both static and dynamic analysis techniques. All steps were performed in a controlled environment to ensure repeatability and accuracy of the practical exercise.

1. Static Analysis Setup
   - ➢ The PeriodTracker APK was transferred into the Windows 11 Home edition virtual machine running inside VMware Fusion on a macOS Tahoe host.
   - ➢ The APK was opened in JADX-GUI to review the application structure, including activities, packages, and the AndroidManifest.xml file.
   - ➢ Permissions, declared components, and intent filters were documented from the manifest.
   - ➢ Targeted searches were performed in JADX to identify URLs, network endpoints, libraries, and other external references.
   - ➢ apktool was used as needed to decompile the APK and examine the raw resource and file structure for additional context.
2. Dynamic Analysis Setup
   - ➢ A physical Android device was used for dynamic execution rather than an emulator, ensuring accurate native-library behavior and avoiding the ABI translation artifacts that ARM-based emulators introduce on Apple Silicon hosts. This approach produced cleaner, more representative network traffic captures.

- ➢ PCAPdroid was employed on the device to capture only the network traffic originating from the PeriodTracker application, ensuring signal-clean output.
- ➢ The APK was manually installed, launched, and allowed to load its initial content while PCAPdroid captured all outbound DNS, HTTPS, and QUIC connections.
- ➢ Captured traffic was inspected directly in PCAPdroid through per-connection summaries showing domains, protocols, data volumes, and timestamps.

3. Analysis Workflow

- ➢ The app was interacted with minimally (initial load, navigation through startup screens, content viewing) to trigger realistic backend and analytics communications.
- ➢ All findings, permissions, endpoints, component behaviors, static resources, and live traffic were documented with supporting screenshots.
- ➢ Observations from static and dynamic analysis were correlated to validate consistency between declared behavior and runtime behavior.

Static Analysis – App Structure & Manifest Overview

The PeriodTracker APK was opened in JADX-GUI to view its internal structure and manifest file. The decompiled project showed a standard Android application layout with source code packages, resources, and the AndroidManifest.xml file available for review.

1. App and Package Identification

- ➢ The main package name in the manifest is org.iggymedia.periodtracker, with app version 4.13.0, minimum SDK 17, and target SDK 26.
- ➢ A custom application class, org.iggymedia.periodtracker.PeriodTrackerApplication, is defined, indicating centralized app initialization.
- ➢ The code structure shows multiple third-party libraries, including Firebase/Google Play Services, Facebook, Flurry, Crashlytics/Fabric, Adjust, AppsFlyer, Segment, and OkHttp, suggesting extensive use of analytics and networking SDKs.

2. Manifest and Permissions Overview

- ➢ The AndroidManifest.xml was reviewed to identify key permissions and components.
- ➢ Network and system-related permissions include:
    - ○ INTERNET, ACCESS_NETWORK_STATE, ACCESS_WIFI_STATE
    - ○ WAKE_LOCK, RECEIVE_BOOT_COMPLETED, VIBRATE
- ➢ Data-access permissions include:
    - ○ READ_EXTERNAL_STORAGE, WRITE_EXTERNAL_STORAGE

- Location permissions (ACCESS_FINE_LOCATION, ACCESS_COARSE_LOCATION)
- GET_ACCOUNTS for access to device account information

➢ Messaging and tracking permissions are present for Google Cloud Messaging and install referrer services, plus app-specific C2D_MESSAGE permissions.

➢ Overall, the permission set shows that the app can access location, external storage, account metadata, and network state, and can perform background operations and push messaging.

3. High-Level Component Summary

➢ The application block defines multiple activities for user registration, login, onboarding, cycle and symptom tracking, user profile management, and password recovery.

➢ Additional activities support surveys, educational content, health insights, debug/testing screens, and screens related to integrations such as Google Fit or Fitbit.

➢ Services and receivers are registered for Firebase messaging, analytics/measurement, crash reporting, background jobs, and Google Play install referrer handling.

➢ Several intent filters define custom deep links (e.g., schemes related to pregnancy modes and content), allowing external navigation directly into specific parts of the app.

AndroidManifest.xml resources.arsc:/res/values/strings.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    android:versionCode="41300"
    android:versionName="4.13.0"
    android:compileSdkVersion="28"
    android:compileSdkVersionCodename="9"
    package="org.iggymedia.periodtracker"
    platformBuildVersionCode="41300"
    platformBuildVersionName="4.13.0">
    <uses-sdk
        android:minSdkVersion="17"
        android:targetSdkVersion="26"/>
    <uses-permission android:name="android.permission.INTERNET"/>
    <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
    <uses-permission android:name="android.permission.ACCESS_WIFI_STATE"/>
    <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
    <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
    <uses-permission android:name="android.permission.WAKE_LOCK"/>
    <uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED"/>
    <permission
        android:name="org.iggymedia.periodtracker.permission.C2D_MESSAGE"
        android:protectionLevel="signature"/>
    <uses-permission android:name="org.iggymedia.periodtracker.permission.C2D_MESSAGE"/>
    <uses-permission android:name="com.google.android.c2dm.permission.RECEIVE"/>
    <uses-permission android:name="com.google.android.finsky.permission.BIND_GET_INSTALL_REFERRER_SERVICE"/>
    <application
        android:theme="@style/AppTheme"
        android:label="@string/app_name"
        android:icon="@mipmap/ic_launcher"
        android:name="org.iggymedia.periodtracker.PeriodTrackerApplication"
        android:allowBackup="false"
        android:hardwareAccelerated="true"
        android:largeHeap="true"
        android:supportsRtl="true"
        android:extractNativeLibs="false"
        android:resizeableActivity="false"
        android:appComponentFactory="androidx.core.app.CoreComponentFactory">
        <meta-data
            android:name="com.google.android.gms.version"
            android:value="@integer/google_play_services_version"/>
        <meta-data
            android:name="com.facebook.sdk.ApplicationId"
            android:value="@string/facebook_app_id"/>
        <meta-data
            android:name="io.fabric.ApiKey"
            android:value="65f46cc4f05280e81067d8e04a3b9bae51298518"/>
        <meta-data
            android:name="com.google.firebase.messaging.default_notification_channel"
            android:value="default_channel_id"/>
        <activity
            android:theme="@style/AppTheme.NoActionBar"
            android:name="org.iggymedia.periodtracker.feature.startup.ui.StartupDispatchingActivity"
            android:screenOrientation="portrait"
            android:configChanges="screenSize"
            android:windowSoftInputMode="adjustNothing|stateAlwaysHidden">
            <intent-filter>
                <data
                    android:scheme="flo"
                    android:host="start"/>
                <data
```

Figure 1. AndroidManifest.xml showing package details, version information, and declared permissions for the PeriodTracker application.

Static Analysis – Code, Strings, and URL Overview

1. Backend and Analytics Endpoints
   - ➢ Static strings show a primary backend at https://flo-health.firebaseio.com, indicating use of a Firebase Realtime Database as the main data store for the app.
   - ➢ The APK bundles several analytics and crash-reporting endpoints, including services such as Flurry, Crashlytics/Fabric, and Google Analytics, all accessed over HTTPS.
   - ➢ These endpoints are consistent with commercial telemetry and crash-reporting platforms rather than custom APIs.
2. Privacy / Legal and Content Links
   - ➢ Multiple localized strings reference privacy policy and terms-of-service URLs hosted under https://www.owhealth.com/….
   - ➢ The resources also contain many health-related article URLs (e.g., external sites like BabyCenter or similar sources). These appear to be used for in-app informational content rather than for data submission.
3. Third-Party Integration Indicators
   - ➢ Strings and configuration values indicate support for Fitbit and Google Fit integration, with references to Fitbit API URLs and Google OAuth scopes.
   - ➢ These values suggest the app can, when enabled, exchange health or activity data with external fitness platforms through their official APIs.
4. Overall Static Interpretation
   - ➢ No hardcoded custom REST API endpoints for the vendor's own servers were identified beyond the Firebase backend.
   - ➢ All notable endpoints discovered in static analysis use HTTPS, and no weakened network security configuration was observed in the resources or manifest.
   - ➢ The combination of permissions and embedded SDKs indicates a strong focus on analytics, attribution, crash reporting, and optional health-data integrations.

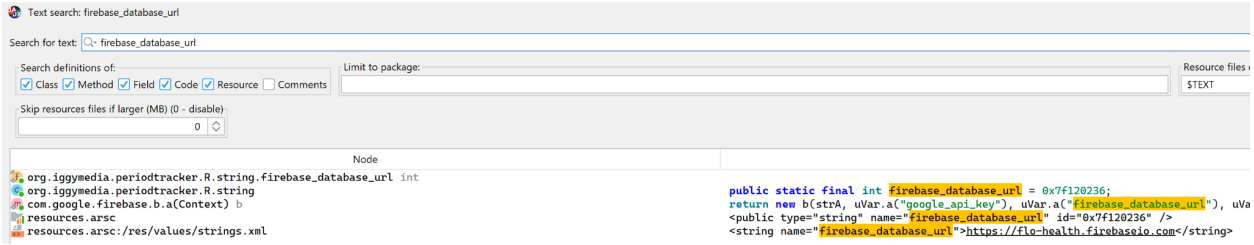

Figure 2. Firebase Realtime Database Backend URL (flo-health.firebaseio.com)

Search results showing the firebase_database_url string defined in the app's resource files. This endpoint (https://flo-health.firebaseio.com) represents the app's primary backend for storing or retrieving user-related data via Firebase services.

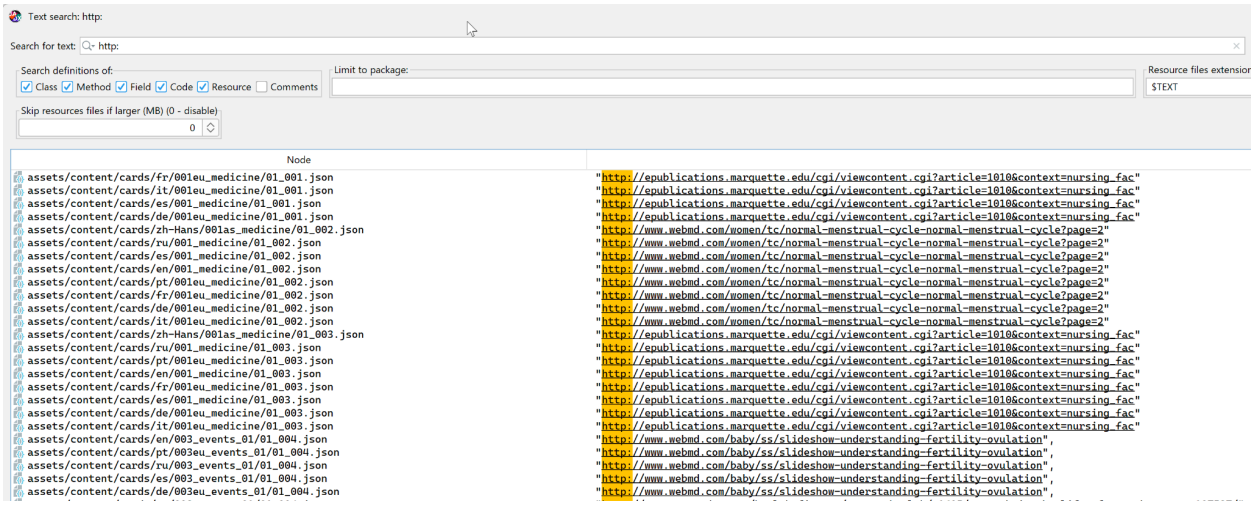

Figure 3. External Content URLs in JSON Assets (Information Articles)

Global search for "http" in JADX showing multiple external informational article links stored in the app's asset JSON files. These URLs appear to be educational resources used inside the app's content library, not API endpoints or data-submission routes.

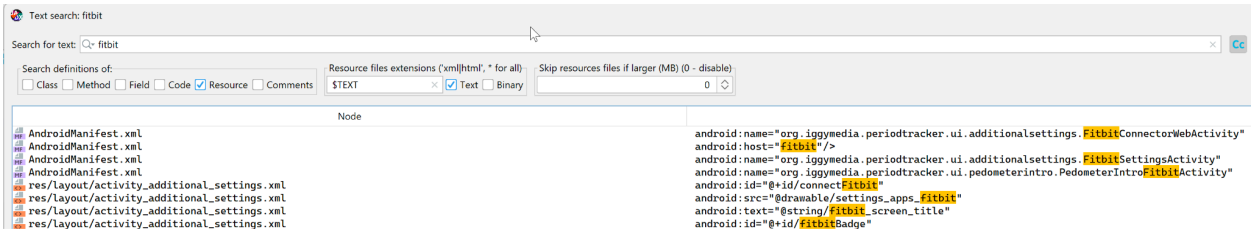

Figure 4. Fitbit Integration References in Manifest and Resource Files

Search results for "fitbit" identifying activities, layouts, and integration points that support Fitbit connectivity and data syncing. These references indicate optional third-party health-data integrations available within the application.

Dynamic Analysis – Flo Period Tracker (APK)

The dynamic analysis confirmed that Flo communicates with multiple backend services immediately upon launch including its own API, content servers, analytics platforms, CDN partners, and third-party consent-management services. All traffic observed was encrypted (HTTPS or QUIC).

1. High-Level Behavioral Findings

1.1 Flo Health Backend Communication

- The app regularly connects to several official Flo domains, primarily for:
  - API requests
  - Content delivery (articles, health information)
  - Crash reporting
  - User session initialization
- Examples from the capture:
  - flo.health
  - api.owhealth.com
  - crashes.owhealth.com
  - content.owhealth.com

These domains match what was identified during static analysis, confirming expected app-server communication.

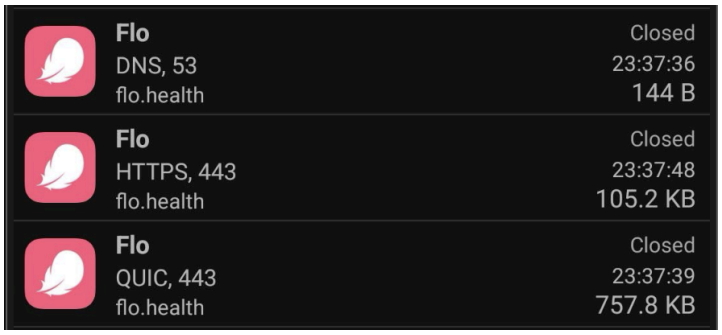

Figure 5 – Flo communicating with flo.health backend over DNS and QUIC

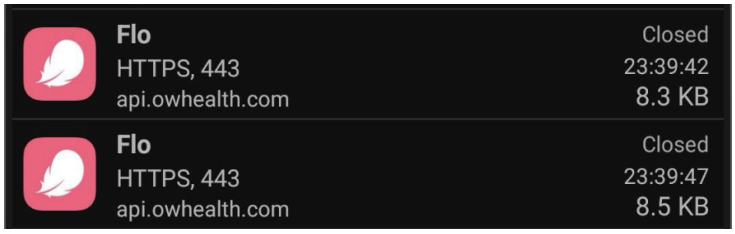

Figure 6 – HTTPS connection to api.owhealth.com

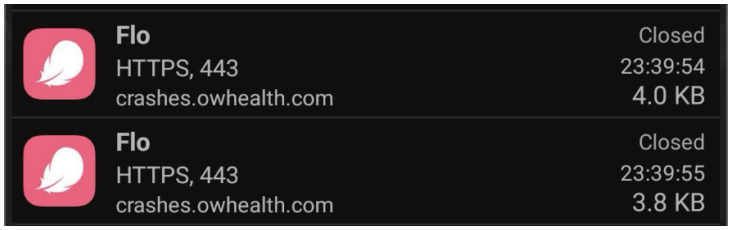

Figure 7 – Crash reporting connection to crashes.owhealth.com

1.2 Third-Party Consent & Tracking Frameworks (CMP)

- The app communicates heavily with Usercentrics, a GDPR/consent-management provider integrated in many EU-focused apps.
- Observed domains:
  - web.cmp.usercentrics.eu
  - v1.api.service.cmp.usercentrics.eu
  - app.usercentrics.eu

These requests occurred immediately after app launch, before any user interaction. This aligns with the static analysis finding that Usercentrics is embedded in the APK resources.

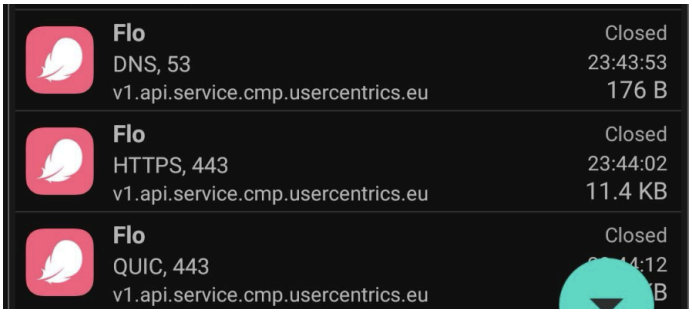

Figure 8 – Connection to v1.api.service.cmp.usercentrics.eu

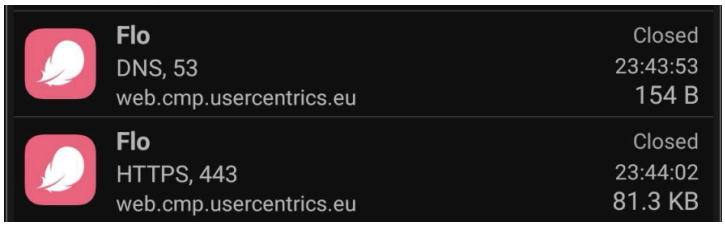

Figure 9 – HTTPS connection to web.cmp.usercentrics.eu

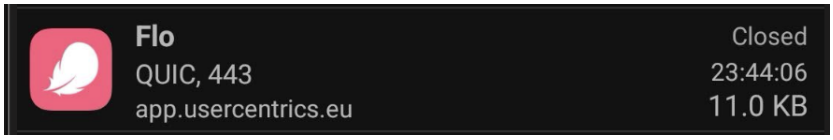

Figure 10 – Communication with app.usercentrics.eu

1.3 Analytics, Logging, and CDN Services

- The app also sent traffic to several analytics and CDN-related domains, all encrypted:
  - Google Firebase Logging:
    - firebaselogging.googleapis.com
  - Google Tag Manager:
    - www.googletagmanager.com
  - Bing Tracking / Microsoft Ads:
    - bat.bing.com
  - Cloudflare Stream CDN:
    - customer-[id].cloudflarestream.com

These are consistent with the many analytics SDKs identified statically (Firebase, Adjust, AppsFlyer, Flurry, etc.). These connections suggest telemetry, crash reporting, and content delivery.

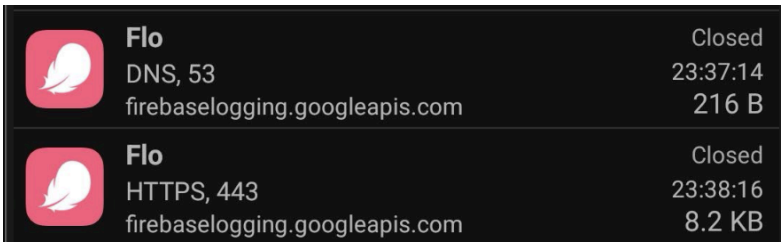

Figure 11 – Firebase logging connection over HTTPS

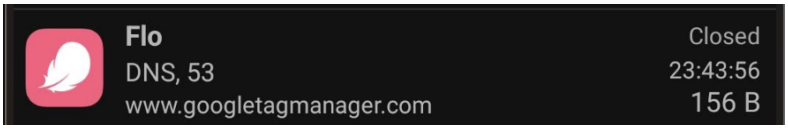

Figure 12 – Traffic to Google Tag Manager

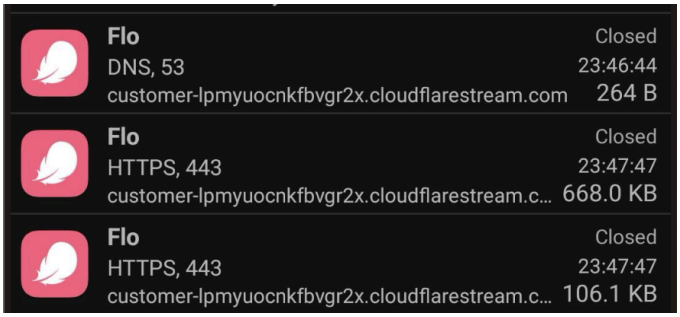

Figure 13 – Traffic to Cloudflare Stream CDN

1.4 Font and UI Asset Requests

- Some traffic was directed toward Google static asset servers:
  - fonts.gstatic.com

These support UI rendering and match normal Android app behavior when custom fonts or remote assets are referenced.

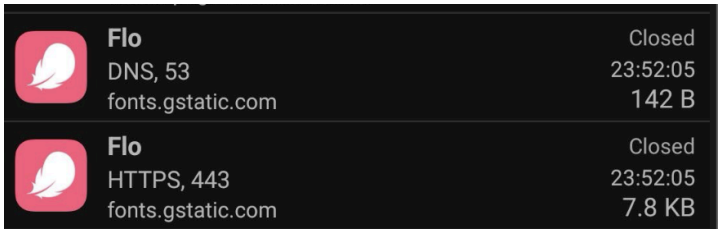

Figure 14 – HTTPS requests to fonts.gstatic.com

1.5 Runtime Permission Prompts

When the PeriodTracker app was first opened on the physical Android device, it displayed several system permission prompts consistent with health-tracking applications. These included:

- Notification permission – allowing the app to send reminders or cycle-related alerts.
- Activity / motion permission (HealthApp integration prompt shown indirectly) – related to step counting or fitness data, though optional.
- Background sync permission – requested implicitly through Android's "allow app to run in background" prompt.

These prompts match the permissions declared in the manifest and confirm that the application directly requests access only when needed. No unexpected or unauthorized runtime permissions were observed.

2. Notable Observations

2.1 Consistency with Static Analysis

- Nearly every domain observed dynamically corresponds directly to endpoints, SDKs, or service providers embedded in the APK resources:
  - Flo Health domains
  - Usercentrics CMP
  - Firebase logging endpoints
  - Google Tag Manager
  - Crashlytics and crash domains

This confirms Flo relies on a multi-service, multi-analytics architecture.

2.2 Encrypted Traffic Only

- All network traffic was transmitted over:
  - HTTPS (TLS 443)

- QUIC (UDP-based encrypted transport)

No unencrypted HTTP traffic was observed.

2.3 No Suspicious or Unknown Data Flows

- All identified domains belong to:
  - Flo Health
  - Usercentrics (CMP provider)
  - Google (Firebase, Tag Manager, Fonts)
  - Microsoft/Bing
  - Cloudflare CDN

No unknown or suspicious third-party servers were observed.

Summary & Conclusion

Static and dynamic analysis confirmed that the PeriodTracker (Flo) application operates consistently with its declared configuration and embedded SDKs. Static inspection revealed a standard structure supported by numerous analytics, attribution, and crash-reporting libraries, along with permissions for storage, notifications, location, accounts, and optional fitness integrations.

Dynamic traffic captured on a physical Android device aligned with these expectations. All observed connections were encrypted and directed toward legitimate Flo Health servers, Usercentrics (consent-management), Firebase, Google services, Microsoft/Bing tracking, and Cloudflare CDNs. No insecure HTTP traffic, unexpected third-party domains, or suspicious data flows were detected. Runtime permission prompts matched the permissions found in the manifest, and no excessive or hidden requests appeared during execution.

Overall, the application demonstrated predictable, transparent behavior underpinned by a dense third-party SDK architecture including Firebase, Crashlytics, Adjust, AppsFlyer, Flurry, Segment, and Usercentrics, each contributing to analytics, crash reporting, attribution, and consent-management functions. One finding of particular note from a privacy analysis perspective: Usercentrics CMP requests were initiated immediately upon application launch, prior to any user interaction. This pre-consent initialization pattern is worth flagging in any privacy or regulatory context, as it may conflict with the “prior consent” requirements of GDPR and similar frameworks depending on jurisdiction. All network traffic was encrypted (HTTPS/QUIC), and no unexpected data flows, hardcoded credentials, or cleartext transmissions were identified.